# Vision Tasks - Detection and Segmentation

## One photo, four questions

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

# The belt camera sees ten fruit

The [51] classifier grades **one** fruit per photo. But the real camera frame looks like this - a whole stall of them.

**Predict:** feed this photo to the [51] classifier. What comes out?

# The belt camera sees ten fruit

The [51] classifier grades **one** fruit per photo. But the real camera frame looks like this - a whole stall of them.

**Predict:** feed this photo to the [51] classifier. What comes out?

**One grade for the whole stall** - say "market", with confidence. For ten fruit at once, that means nothing: *which* fruit? *where*? *how many*?

We need models whose **output has structure**: boxes, masks. That is today - and it closes the chapter.



# Outline

One photo, four tasks

Object detection

Building a detector

Segmentation

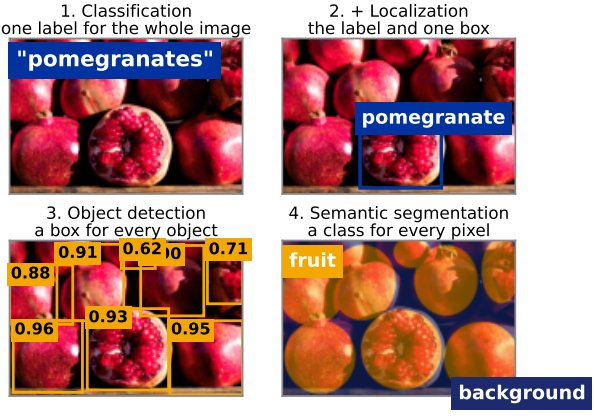
The 2026 toolbox

The sorting line, completed - and the chapter close

# One photo, four tasks

What changes is the *output*, not the backbone.

# The task zoo




Same photo, same conv backbone - four **output structures**: a label, a label + a box, a box *per object*, a class *per pixel*. The output structure *defines* the task. (Overlays drawn for illustration - no model ran on this photo.)

# Object detection

What *and* where - a class and a box for every object in the frame.

# From "what" to "what and where"

For *one* object, no new machinery is needed - just a **longer label vector**:

$$\mathbf{y} = \begin{bmatrix} b_x \\ b_y \\ b_h \\ b_w \\ c_o \\ c_{\text{pom}} \\ c_{\text{apple}} \\ c_{\text{quince}} \end{bmatrix} \begin{matrix} \text{box center } x \\ \text{box center } y \\ \text{box height} \\ \text{box width} \\ \text{object present? (binary)} \\ \text{class, one-hot} \\ \text{class, one-hot} \\ \text{class, one-hot} \end{matrix}$$

Classification **and** box regression, one network: the box coordinates are just four regression outputs ([01]'s squared loss next to [11]'s log-loss).

Box centre and size are **fractions of the photo** (0 to 1), so the same numbers work for any image size.

One entry looks odd: $c_o$, "is there an object at all?" Why would a fruit grader need a "no fruit" answer? Next slide.

# "Nothing here" has to be an answer

**Predict:** the belt runs empty for a second - no fruit in the frame. What does the network output?

# "Nothing here" has to be an answer

**Predict:** the belt runs empty for a second - no fruit in the frame. What does the network output?

Without $c_o$ it *cannot* say "none": every output is a box and a class, so it **invents a fruit** somewhere.

With $c_o$, empty photos go into training too, labeled $c_o = 0$:

pomegranate: $\mathbf{y} = (.4, .6, .3, .2, \mathbf{1}, 1, 0, 0)$
empty belt: $\mathbf{y} = (\,?,\ \ ?,\ \ ?,\ \ ?,\ \ \mathbf{0}, ?, ?, ?)$

The "?" entries are **don't care**: there is no box to learn, so the loss checks only $c_o$.

**Confidence** = the network's *predicted* $c_o$: a number from 0 to 1, "how sure am I that an object is here?" - [11]'s predicted probability, for the yes/no question "object or not". Every box a detector outputs carries one.

# "Nothing here" has to be an answer

**Predict:** the belt runs empty for a second - no fruit in the frame. What does the network output?

Without $c_o$ it *cannot* say "none": every output is a box and a class, so it **invents a fruit** somewhere.

With $c_o$, empty photos go into training too, labeled $c_o = 0$:

pomegranate: $\mathbf{y} = (.4, .6, .3, .2, \mathbf{1}, 1, 0, 0)$

empty belt: $\mathbf{y} = (?, ?, ?, ?, \mathbf{0}, ?, ?, ?)$

The "?" entries are **don't care**: there is no box to learn, so the loss checks only $c_o$.

**Confidence** = the network's *predicted* $c_o$: a number from 0 to 1, "how sure am I that an object is here?" - [11]'s predicted probability, for the yes/no question "object or not". Every box a detector outputs carries one.

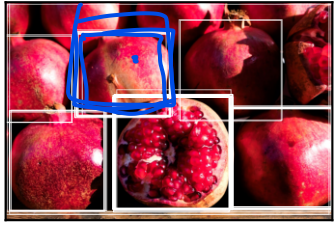
**The real reason:** a photo can hold **any number** of fruit, but a network has a **fixed output size**. The fix: give it many output *slots*, each with its own $c_o$. Slots with a fruit fill in a box; every other slot answers $c_o = 0$, "nothing here". That is a detector.

# Why a detector answers with so many boxes

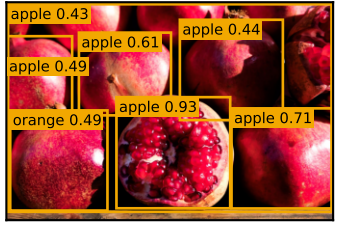
1. raw: 401 boxes score > 5%



2. confidence > 0.4: 45 boxes



3. after NMS: 7 boxes




1. The slots from the last slide: a fixed set of **candidate** boxes, each with a **confidence**. This model: 3,234 for every photo. YOLO v1: 98. YOLOv8: 8,400.
2. Most candidates look at background: low confidence, cut by a **threshold**.
3. A fruit covers many positions, and each nearby candidate finds it: **duplicates**. NMS keeps the best one per object (two slides on).

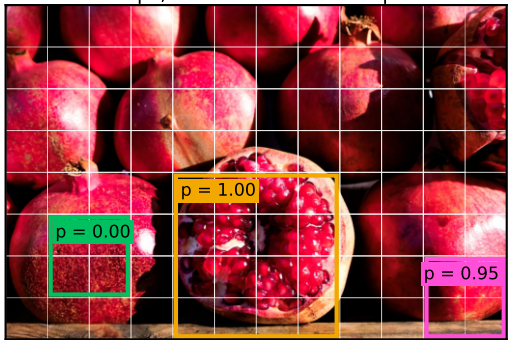
A real detector (SSDlite, trained on COCO) on our photo. COCO has no pomegranate class, so it says "apple" - and boxes the whole crate as one more apple.

The many boxes are the **raw** output, not the answer. The answer is what survives the threshold and NMS.



# Where candidates come from - idea 1: crop and classify

120 crops, 3 sizes: 120 forward passes




$p$ = ImageNet ResNet-18's probability of "pomegranate" for that crop. White grid: the smallest windows, overlapping by half.

The obvious detector: slide a **window** over the photo at several sizes, crop, and run a **classifier** on every crop. A high score means "fruit here" - the window *is* the box.

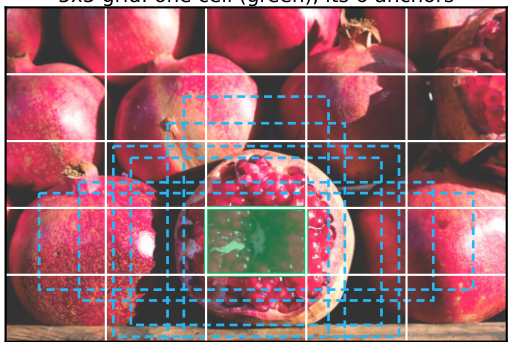
Measured on our photo: **120 crops, 120 forward passes**, 12 s on a laptop CPU. Real detectors need more sizes, shapes and positions: thousands of crops.

**R-CNN** (2014) did exactly this, on ∼**2,000** smart crops per photo: **13 s per image** on a GPU.

So yes: the first detectors ran one model on a ton of boxes. The waste: overlapping crops recompute the same pixels again and again.

# Idea 2: one pass, a small head at every cell

5x5 grid: one cell (green), its 6 anchors



The real model from the many-boxes slide (SSDlite). Dashed: preset box shapes, **anchors**, centred on the green cell.

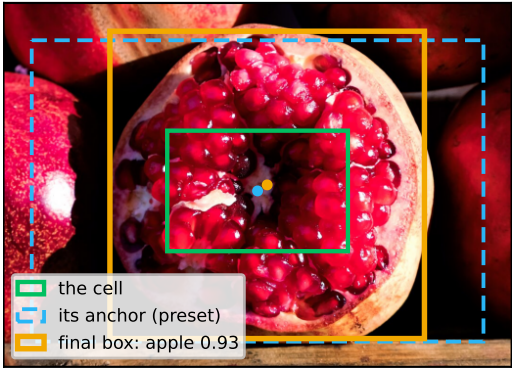
Run the backbone **once**. Its last maps form a grid - like [51]'s $7 \times 7 \times 512$ - and each cell's vector describes the photo *around* that cell.

Put a small conv layer, the **head**, on the grid. A conv slides by construction ([48]), so the head runs at **every cell** in the same pass, reusing the backbone's work.

Per cell, for each of its 6 anchors: **4 box numbers** + **91 class scores** (one class is "background": SSD's "nothing here").

6 grids, fine to coarse ($20\times20$ for small fruit ... $1\times1$ for the whole photo): $(400+100+25+9+4+1)\times6 = \mathbf{3{,}234}$ candidates.

# Box sizes are not fixed: anchors + offsets




The top candidate on our photo, zoomed. COCO has no pomegranate class.

An anchor is only a **starting guess**. The head **regresses** ([01]) how to fix it: shift the centre, stretch the width and height,

$$w = w_{\text{anchor}} \cdot e^{t_w}, \qquad h = h_{\text{anchor}} \cdot e^{t_h}$$

($e^t$: any stretch factor, never negative).

This fruit, measured: anchor $160 \times 160$ on the model's $320 \times 320$ input; the head said **move 3 px, stretch** $\times 0.70$ **and** $\times 1.02 \rightarrow$ a $111 \times 163$ box, "apple" 0.93.

No fixed box size: the anchor sets roughly where and how big, the network fixes the rest. And no box has to fit inside its cell - this one is $1.7 \times 2.5$ cells.

# IoU: the overlap score

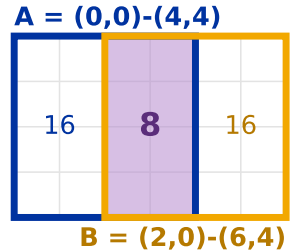
Detectors output many boxes, so we constantly ask: **how much do two boxes agree?**

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

$1$ = identical boxes, $0$ = no overlap; in practice IoU $\geq 0.5$ usually counts as "the same object".

Worked example: $A = (0,0)$-$(4,4)$, $B = (2,0)$-$(6,4)$. Intersection $= 8$, union $= 16 + 16 - 8 = 24$, so IoU $= 8/24 = \mathbf{1/3}$.

areas: A 16, B 16, overlap 8




intersection 8, union 16 + 16 - 8 = 24
IoU = 8 / 24 = 1/3

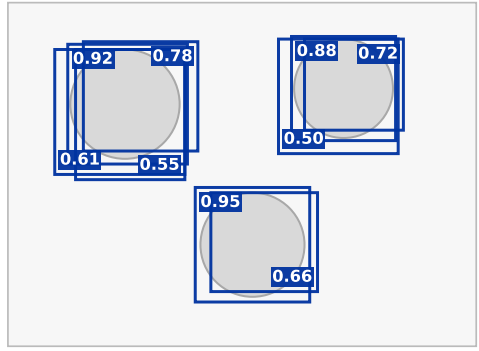
# NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

**Keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45), repeat. Too high: duplicates survive. Too low: touching fruit lose a box.

**2026:** the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

9 candidate boxes, 3 objects



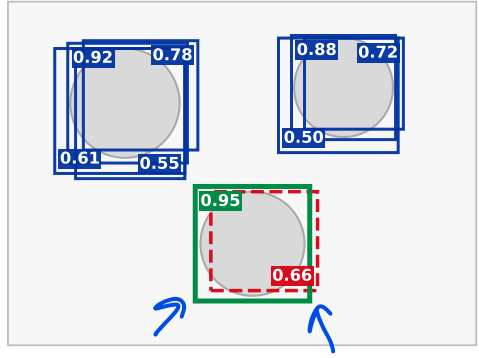
# NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

**Keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45), repeat. Too high: duplicates survive. Too low: touching fruit lose a box.

**2026:** the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

keep 0.95, drop its neighbors (IoU > 0.45)



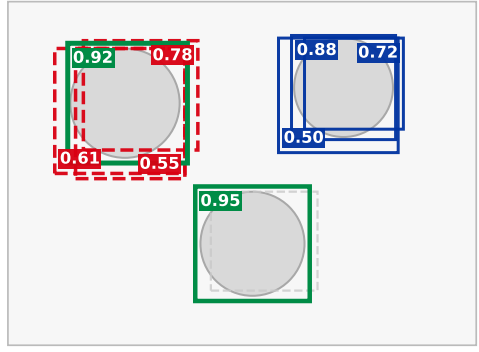
# NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

**Keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45), repeat. Too high: duplicates survive. Too low: touching fruit lose a box.

**2026:** the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

keep 0.92, drop its neighbors (IoU > 0.45)



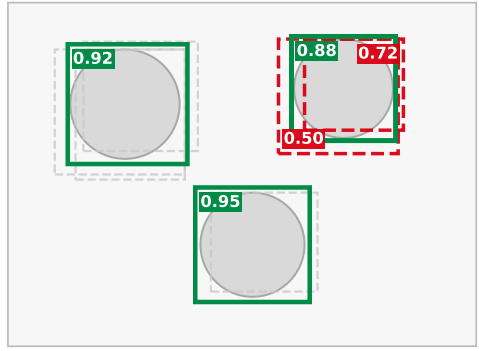
# NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

**Keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45), repeat. Too high: duplicates survive. Too low: touching fruit lose a box.

**2026:** the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

keep 0.88, drop its neighbors (IoU > 0.45)



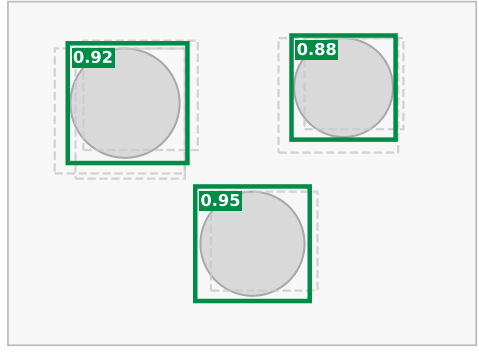
# NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

**Keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45), repeat. Too high: duplicates survive. Too low: touching fruit lose a box.

**2026:** the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

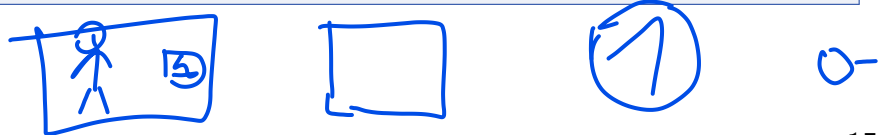
3 survivors: one box per object



# Scoring a detector: mAP in one line

- A predicted box is **correct** if the class matches and its IoU with a true box passes a threshold (say 0.5).
- That makes every prediction a TP or FP - so **precision** and **recall** are exactly [12]'s, computed over boxes.
- **AP** = area under the precision-recall curve, per class; **mAP** = the mean over classes. Benchmarks also sweep the IoU threshold (0.5 to 0.95) and average once more.

No new metric theory: mAP is the **[12] machinery wrapped around boxes**, swept over IoU thresholds.



# mAP by hand

One class, **2** true pomegranates in the photo. The detector returns 4 boxes; sort them by score and mark each **TP** (IoU $\geq 0.5$ with a not-yet-matched true box) or **FP**:

| rank | score | | precision | recall |
|---|---|---|---|---|
| 1 | 0.9 | TP | $1/1 = 1.00$ | $1/2 = 0.50$ |
| 2 | 0.8 | FP | $1/2 = 0.50$ | 0.50 |
| 3 | 0.7 | TP | $2/3 = 0.67$ | $2/2 = 1.00$ |
| 4 | 0.6 | FP | $2/4 = 0.50$ | 1.00 |

**AP** = area under the precision-recall steps - each TP adds its recall gain times its precision:

$$\text{AP} = 0.5 \times 1.00 + 0.5 \times 0.67 = \mathbf{0.83}$$

**mAP** = the mean over classes: if apples score 0.60, $\text{mAP} = (0.83 + 0.60)/2 \approx 0.72$.

**Ranking matters**: the same four boxes with the two false alarms scored highest give $\text{AP} = 0.5 \times \frac{1}{3} + 0.5 \times \frac{1}{2} \approx 0.42$ - half the score. mAP rewards finding every object *and* being most confident about the right ones. (COCO then also averages over IoU thresholds 0.50 to 0.95.)

# Two ways to build a detector

You know what a detector outputs and how it is scored. Crop-and-classify and one-pass-over-a-grid were not just ideas: they are the two families of real detectors. One of them became the industry default.

# Two families of detectors

**Two-stage** = idea 1, made fast: first *propose* candidate regions, then classify and refine each.

- R-CNN (2014) → Fast → **Faster R-CNN** (2015)
- accurate, but two passes of work

**One-stage** = idea 2: predict all boxes and classes in *one* forward pass.

- **YOLO** ("You Only Look Once", 2016), SSD (Single Shot MultiBox Detector, 2016)
- real-time - video-rate detection

Two-stage bought **accuracy**, one-stage bought **speed** - and a decade of iteration has mostly closed the gap.

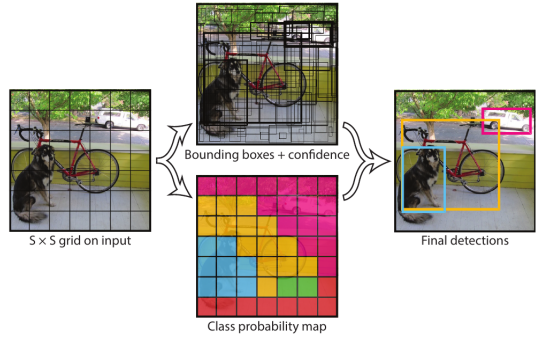
In practice, YOLO became the industry default: you `pip install` a pretrained one and fine-tune it on *your* boxes ([51]'s recipe). That **is** the sorting line's counting camera.

Girshick et al. 2014; Ren et al. 2015; Redmon et al. 2016; Liu et al. 2016.

# How one-stage works: the YOLO grid

- Divide the image into an $S \times S$ grid ($7 \times 7$ in the paper).
- **Each cell** predicts 2 candidate boxes - $(x, y, w, h, \text{confidence})$ - plus class probabilities. Confidence: the predicted $c_o$, trained towards how well the box fits (its IoU with the truth).
- All 98 boxes come out of **one forward pass**, as one $7 \times 7 \times 30$ tensor. That is why it runs in real time.

A flood of overlapping candidates by *design* - exactly why **NMS** exists.




The paper's own diagram (Redmon et al., "You Only Look Once", 2016).



# Meet YOLO's author

**Joseph Redmon**, PhD student at the University of Washington: YOLO (2016), YOLO9000 (2017) and YOLOv3 (2018), all in his own C framework, Darknet.

His CV (right) is a My Little Pony fan page: "This little pony went to Middlebury College..."

The YOLOv3 paper opens: *"Sometimes you just kinda phone it in for a year, you know? I didn't do a whole lot of research this year. Spent a lot of time on Twitter."* It cites Newton's *Principia* for the word "momentum".

The serious end: in 2020 he quit computer vision - "the military applications and privacy concerns eventually became impossible to ignore." YOLO v4 onward is other people's work; today mostly Ultralytics'.



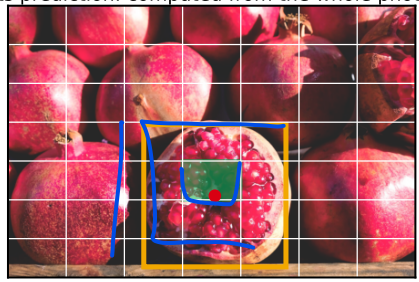

pjreddie.com/static/resume.pdf

# A cell is an address, not a window

the cell's own patch, alone



its prediction: computed from the whole photo

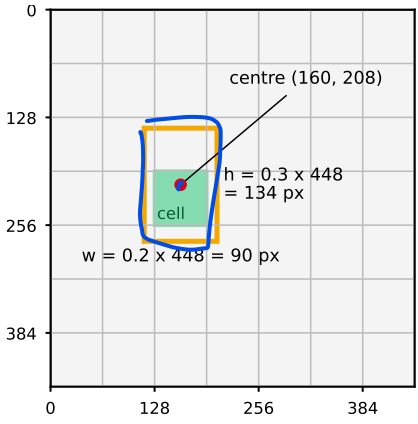


Alone, the cell's patch (left) is a slab of seeds - like the dog's belly in the paper's photo. Nobody could tell where the fruit ends.

But YOLO v1 ends in two **fully connected** layers: every number of every cell is computed from **every pixel**.

The grid decides **who reports** an object: the cell holding its centre. The whole network decides **what** gets reported - so the box can be far bigger than the cell.

# Decoding one YOLO cell, by hand

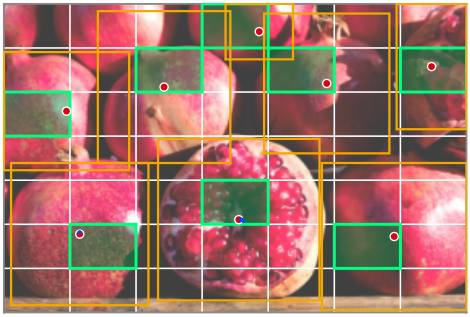



YOLO squashes every photo to $448 \times 448$: 49 cells of 64 px. One box of the green cell (row 3, column 2) outputs:

| | |
|---|---|
| $x = 0.5,\ y = 0.25$ | centre, as fractions of the **cell** |
| $w = 0.2,\ h = 0.3$ | size, as fractions of the **image** |
| confidence 0.8, | $P(\text{pomegranate} \mid \text{object}) = 0.9$ |

- centre: $\big((2 + 0.5) \cdot 64,\ (3 + 0.25) \cdot 64\big) = (160, 208)$
- size: $90 \times 134$ px - wider and taller than the 64 px cell
- score: $0.8 \times 0.9 = \mathbf{0.72}$, the number NMS sorts by

# Training the grid: who learns what



Orange: true boxes. Green: the cell holding a fruit's centre (red dot).

A fixed-size output learning from **any number** of objects: each true box goes into its responsible slot; every other slot learns "nothing here".

YOLO's rule: the cell holding a fruit's **centre** is responsible for it - and of its two predictors, the one that already overlaps it best.

On this photo: 8 fruit $\rightarrow$ **8** responsible predictors of 98.

- those 8: box loss, class loss, confidence $\rightarrow$ their IoU with the truth;
- the other **90**: confidence $\rightarrow$ 0, box "don't care".

**90 "nothing" vs 8 "fruit"**: [12]'s class imbalance. The empty cells' gradients made training diverge, so YOLO weights them by $\lambda_{\text{noobj}} = 0.5$ (box errors by $\lambda_{\text{coord}} = 5$). Later: the **focal loss** (Lin et al., 2017) shrinks the loss of easy background.

# The detection loss: regression and classification, added

**Predict:** a detector outputs box numbers (regression) *and* class scores (classification). Which loss does it train with?

# The detection loss: regression and classification, added

**Predict:** a detector outputs box numbers (regression) *and* class scores (classification). Which loss does it train with?

**Both, added.** Each output gets the loss that fits its type; one backward pass trains them all:

| **part** | **which slots** | **YOLO v1 (2016)** | **modern YOLO** |
|---|---|---|---|
| box $x, y, w, h$ | responsible only | squared error ([01]), weight 5 | IoU-based loss |
| confidence | all; empty ones $\times 0.5$ | squared error to IoU or 0 | inside the class scores |
| class | cells with an object | squared error | log-loss ([11]) |

$$L = \lambda_{\text{coord}} L_{\text{box}} + L_{\text{conf}} + L_{\text{class}}$$

# The detection loss: regression and classification, added

**Predict:** a detector outputs box numbers (regression) *and* class scores (classification). Which loss does it train with?

**Both, added.** Each output gets the loss that fits its type; one backward pass trains them all:

| part | which slots | YOLO v1 (2016) | modern YOLO |
|---|---|---|---|
| box $x, y, w, h$ | responsible only | squared error ([01]), weight 5 | IoU-based loss |
| confidence | all; empty ones $\times 0.5$ | squared error to IoU or 0 | inside the class scores |
| class | cells with an object | squared error | log-loss ([11]) |

$$L = \lambda_{\text{coord}} L_{\text{box}} + L_{\text{conf}} + L_{\text{class}}$$

YOLO v1 used squared error even for classes, "because it is easy to optimize". On $\sqrt{w}, \sqrt{h}$: a 10 px miss matters on a small fruit, not on a big crate.

The $\lambda$ weights set how much each part counts - the knob from the last slide.

# Your own detector: label boxes, or prompt for them

**Route 1 - fine-tune a pretrained YOLO on your boxes** ([51]'s recipe; YOLO v1 itself started from ImageNet-pretrained convs). Ultralytics' advice for good results: $\geq$ 1,500 images and $\geq$ 10,000 labeled boxes *per class*.

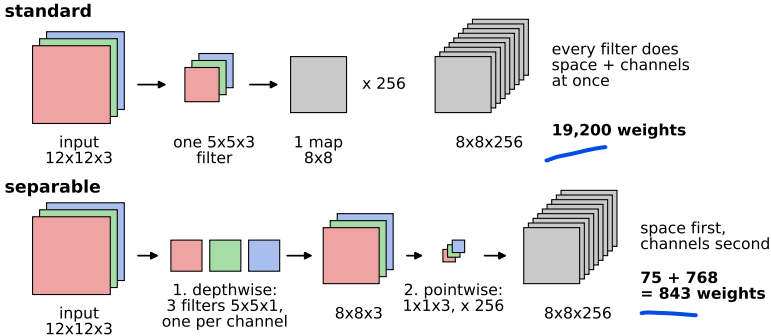
**Label every fruit in every photo.** An unlabeled fruit is not "missing data": its cell is trained to say **"nothing here"**. Half-labeled photos teach the detector to ignore fruit.

**Route 2 - prompt instead of label.** *Open-vocabulary* models take the classes as **text**: YOLO-World (Cheng et al., 2024) detects whatever names you type, no boxes needed; SAM 3 (Meta, Nov 2025) returns a mask for every instance of a typed concept.

**In practice, both:** prompt a big model to *draft* the boxes, people correct them, then fine-tune a small, fast YOLO for the belt camera. [51]'s price of boxes, cut twice.

# Making it fast: separable convolutions

**Why here?** The belt camera needs boxes for every frame - 30 a second, on a small computer, no data centre. The detector from the many-boxes slide, **SSDlite-MobileNetV3**, is phone-sized because it is built from *separable* convolutions.




First each channel is filtered **alone** (space), then $1 \times 1$ convs ([50]) **mix** the channels. Less flexible than a standard filter, far cheaper: next slide.

# Separable convolutions: the count

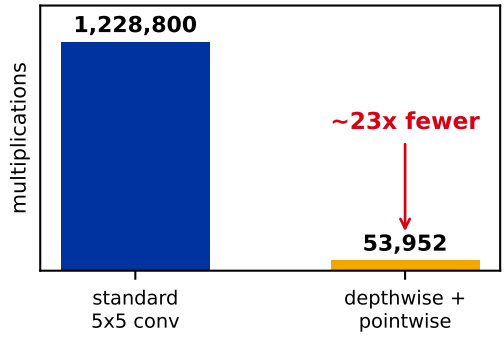
Input $12 \times 12 \times 3$, $5 \times 5$ filters, 256 output channels $\Rightarrow$ output $8 \times 8$. Count multiplications:

**Standard:**

$$256 \cdot \underbrace{(5 \cdot 5 \cdot 3)}_{\text{per stop}} \cdot \underbrace{(8 \cdot 8)}_{\text{stops}} = \mathbf{1{,}228{,}800}$$

**Depthwise + pointwise**:

$$\underbrace{3 \cdot 25 \cdot 64}_{4{,}800} + \underbrace{256 \cdot 3 \cdot 64}_{49{,}152} = \mathbf{53{,}952}$$




$\sim$**23**$\times$ fewer multiplications for the same output shape. This is why a pretrained detector runs *on* the sorting line, not in a cloud.

Not the same as a *spatially* separable kernel (Sobel = column $\times$ row, [48]): that one splits space; this one splits space from channels.

## Segmentation

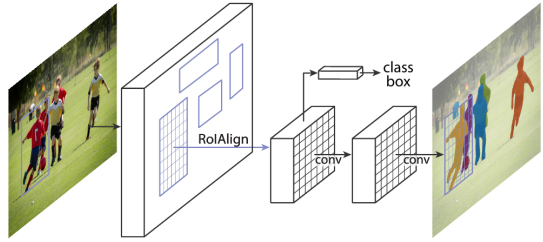
Every pixel gets an answer.



# Semantic vs instance

Segmentation = **per-pixel classification**.

- **Semantic**: every pixel gets a class label - all our pomegranates merge into *one* fruit mask (task-zoo panel 4).
- **Instance**: each fruit gets its **own** mask. **Mask R-CNN** = Faster R-CNN + a small mask head per region (He et al., 2017).
- **Panoptic**: both at once - every pixel labeled *and* grouped into instances.




Mask R-CNN: detection boxes plus a per-region mask branch (He et al., 2017).

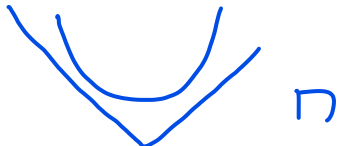
From the [50] epilogue: **SAM 3** (Meta, 2025) made segmentation *promptable* - type a concept, get a mask for every instance.

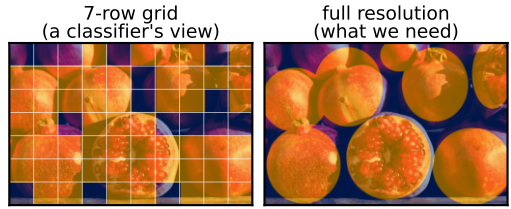
# Segmentation's dilemma: see far, stay sharp

To label *one* pixel, a network needs two things at once:

- **Context**: pomegranate or tomato? Only a wide view can tell - a big receptive field ([50]).
- **Resolution**: the mask must follow the fruit's edge, pixel by pixel.

A classifier buys context by **shrinking**: pooling and stride take ResNet's $224 \times 224$ input down to $7 \times 7$. Fine for one label per photo - as a mask it is the blocky left panel.



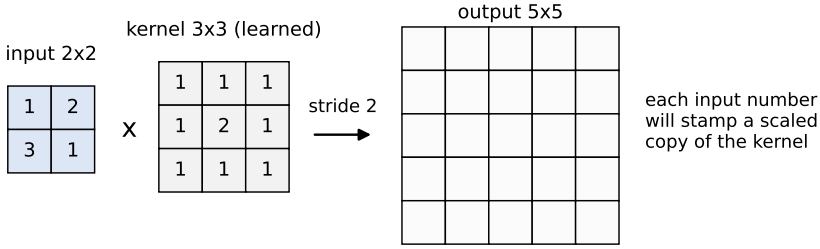



Two ways out - both computed by hand in [48]:

1. **shrink, then grow back**: transposed convolution (next), U-Net;
2. **never shrink**: dilated convolution (after it).

# Transposed convolution: a learnable way up

Convs and pools **shrink** maps; a per-pixel answer needs full resolution back. A **transposed convolution** goes up: each input number **stamps** a scaled copy of the kernel onto a bigger output, and overlapping stamps **add**. **(Click.)**

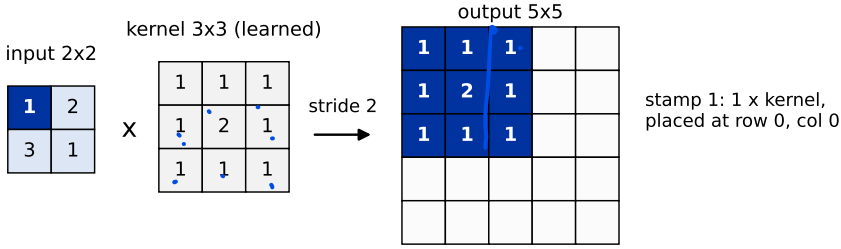



Output size $(2-1)\cdot 2+3=5$. The 9 kernel numbers are **learned**. U-Net uses a $2\times 2$ kernel, stride 2: no overlap, each cell becomes a $2\times 2$ block - exactly double. (Name: a conv is a matrix multiply $\mathbf{y}=K\mathbf{z}$; this one uses $K^{\top}$'s pattern to go back up.)

**Watch-out: checkerboard.** The bold cells got more stamps; on real images that uneven overlap tiles a checker pattern. Fix: kernel divisible by the stride, or upsample then convolve (Odena et al., 2016).

# Transposed convolution: a learnable way up

Convs and pools **shrink** maps; a per-pixel answer needs full resolution back. A **transposed convolution** goes up: each input number **stamps** a scaled copy of the kernel onto a bigger output, and overlapping stamps **add**. **(Click.)**

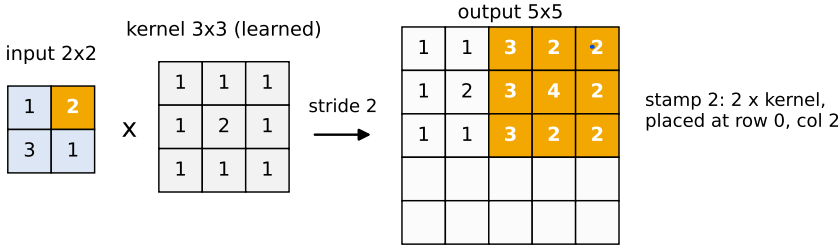



Output size $(2-1)\cdot 2+3=5$. The 9 kernel numbers are **learned**. U-Net uses a $2\times 2$ kernel, stride 2: no overlap, each cell becomes a $2\times 2$ block - exactly double. (Name: a conv is a matrix multiply $\mathbf{y}=K\mathbf{z}$; this one uses $K^\top$'s pattern to go back up.)

**Watch-out: checkerboard.** The bold cells got more stamps; on real images that uneven overlap tiles a checker pattern. Fix: kernel divisible by the stride, or upsample then convolve (Odena et al., 2016).

# Transposed convolution: a learnable way up

Convs and pools **shrink** maps; a per-pixel answer needs full resolution back. A **transposed convolution** goes up: each input number **stamps** a scaled copy of the kernel onto a bigger output, and overlapping stamps **add**. **(Click.)**

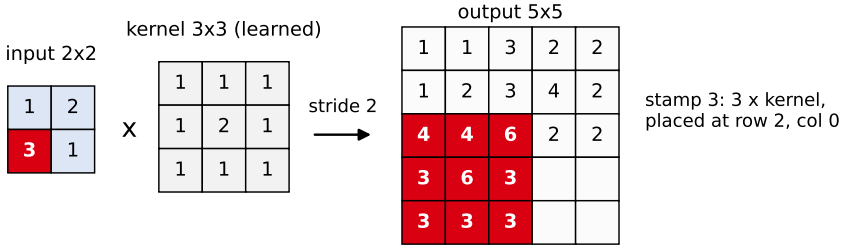



Output size $(2-1)\cdot 2+3=5$. The 9 kernel numbers are **learned**. U-Net uses a $2\times 2$ kernel, stride 2: no overlap, each cell becomes a $2\times 2$ block - exactly double. (Name: a conv is a matrix multiply $\mathbf{y}=K\mathbf{z}$; this one uses $K^{\top}$'s pattern to go back up.)

**Watch-out: checkerboard.** The bold cells got more stamps; on real images that uneven overlap tiles a checker pattern. Fix: kernel divisible by the stride, or upsample then convolve (Odena et al., 2016).

# Transposed convolution: a learnable way up

Convs and pools **shrink** maps; a per-pixel answer needs full resolution back. A **transposed convolution** goes up: each input number **stamps** a scaled copy of the kernel onto a bigger output, and overlapping stamps **add**. **(Click.)**

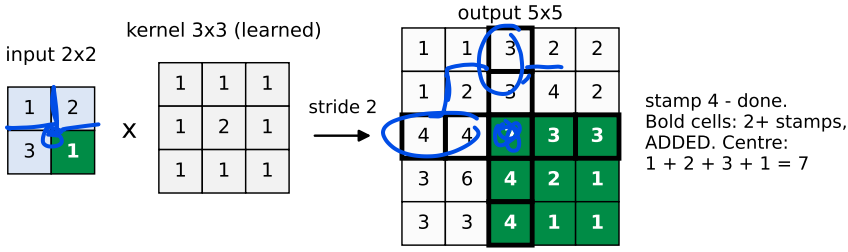



Output size $(2-1)\cdot 2+3=5$. The 9 kernel numbers are **learned**. U-Net uses a $2\times 2$ kernel, stride 2: no overlap, each cell becomes a $2\times 2$ block - exactly double. (Name: a conv is a matrix multiply $\mathbf{y}=K\mathbf{z}$; this one uses $K^{\top}$'s pattern to go back up.)

**Watch-out: checkerboard.** The bold cells got more stamps; on real images that uneven overlap tiles a checker pattern. Fix: kernel divisible by the stride, or upsample then convolve (Odena et al., 2016).

# Transposed convolution: a learnable way up

Convs and pools **shrink** maps; a per-pixel answer needs full resolution back. A **transposed convolution** goes up: each input number **stamps** a scaled copy of the kernel onto a bigger output, and overlapping stamps **add**. **(Click.)**




Output size $(2-1)\cdot 2+3=5$. The 9 kernel numbers are **learned**. U-Net uses a $2\times 2$ kernel, stride 2: no overlap, each cell becomes a $2\times 2$ block - exactly double. (Name: a conv is a matrix multiply $\mathbf{y}=K\mathbf{z}$; this one uses $K^{\top}$'s pattern to go back up.)

**Watch-out: checkerboard.** The bold cells got more stamps; on real images that uneven overlap tiles a checker pattern. Fix: kernel divisible by the stride, or upsample then convolve (Odena et al., 2016).

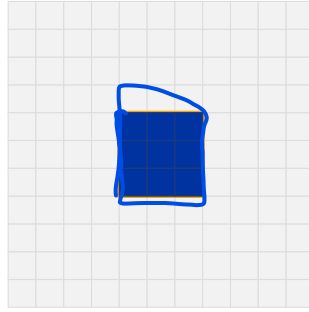
# Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a $3 \times 3$ kernel with dilation $d$ taps cells $d$ apart. **(Click to widen.)**

- Same 9 weights, wider view - no pooling, no lost pixels.
- [50] grew the receptive field by *stacking*; dilation grows it **per layer**.
- Beyond images: **WaveNet** audio and temporal convolutional networks for time series stack dilated 1D convs.

Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.

d = 1: 9 weights span 3x3

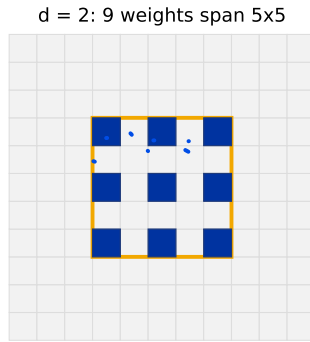


# Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a $3 \times 3$ kernel with dilation $d$ taps cells $d$ apart. **(Click to widen.)**

- Same 9 weights, wider view - no pooling, no lost pixels.
- [50] grew the receptive field by *stacking*; dilation grows it **per layer**.
- Beyond images: **WaveNet** audio and temporal convolutional networks for time series stack dilated 1D convs.

Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.

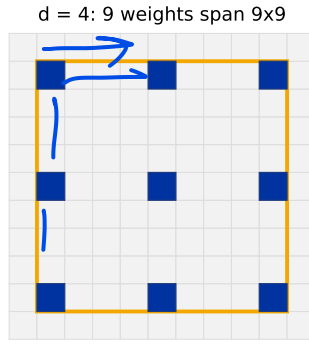


# Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a $3 \times 3$ kernel with dilation $d$ taps cells $d$ apart. **(Click to widen.)**

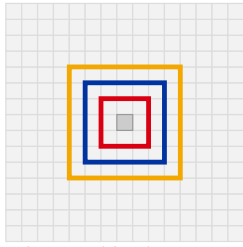
- Same 9 weights, wider view - no pooling, no lost pixels.
- [50] grew the receptive field by *stacking*; dilation grows it **per layer**.
- Beyond images: **WaveNet** audio and temporal convolutional networks for time series stack dilated 1D convs.

Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.



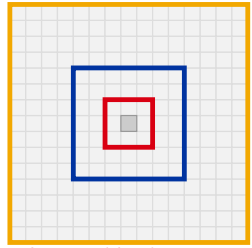
# Stack dilations and the field explodes

standard: grows linearly



layer 1 (d=1): 3x3
layer 2 (d=1): 5x5
layer 3 (d=1): 7x7

dilated: grows exponentially

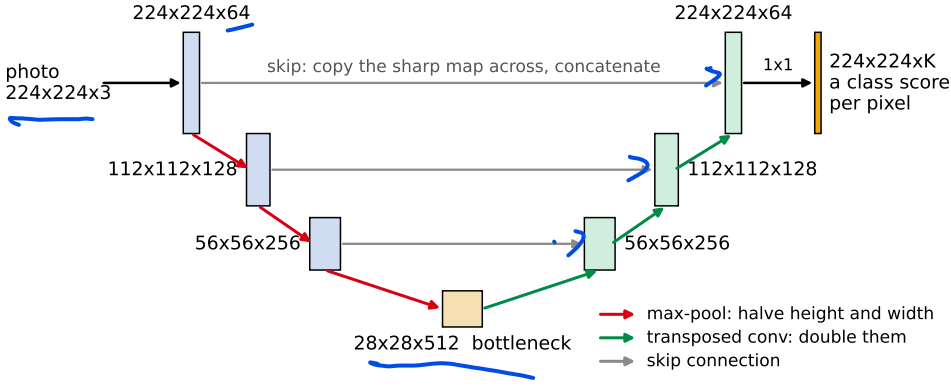


layer 1 (d=1): 3x3
layer 2 (d=2): 7x7
layer 3 (d=4): 15x15

**same cost either way: 9 weights per layer**

Doubling the dilation each layer ($d = 1, 2, 4$) grows the receptive field **exponentially**: $15 \times 15$ instead of $7 \times 7$, for the same nine weights per layer (compare [50]'s receptive-field frame).

# U-Net: the segmentation workhorse

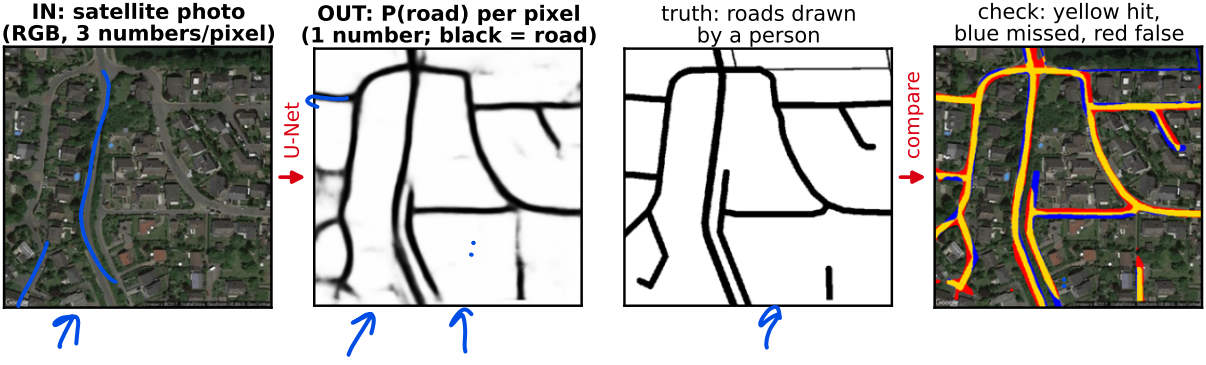



**Down**: pooling trades resolution for context ($224 \to 28$). **Up**: transposed convs climb back to 224. **Skips**: the sharp early maps are *concatenated* onto the upsampled ones - the detail the bottleneck lost.

[50]'s skips again - there they rescued **optimization**, here they carry **resolution**.

Ronneberger et al., 2015 (medical images; four levels deep).

# U-Net in action: roads from the sky




**In:** a satellite tile, $512 \times 512 \times 3$ - a colour photo.
**Out: one number per pixel**, $512 \times 512 \times 1$:
"this pixel is road", as a probability. Loss: [11]'s log-loss, once per pixel, summed.

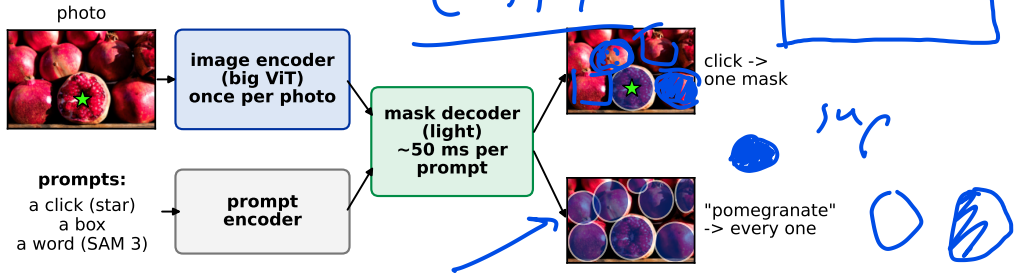
Same size in, same size out - only the channels change ($3 \rightarrow 1$). The check panel is [12]'s confusion matrix, *painted onto the map*.

LMU I2DL, CC BY 4.0, rearranged.

## What people actually use in 2026

Almost nobody trains a detector or a segmenter from scratch now. They start from a few big pretrained models - and one family made "segment this" a single click.

# Segment Anything: one model, any object




**SAM** (Meta, 2023): click or draw a box, get a mask - even for objects it never saw. The heavy encoder runs **once** per photo; each prompt costs only the light decoder, ∼**50 ms on a CPU**: it feels instant. Its 1.1 billion training masks were drafted by SAM and fixed by people ([51]).

**SAM 2** (2024): video - click once, the mask follows the object.

**SAM 3** (Nov 2025): type "pomegranate", get a mask for **every** one - detection and segmentation by prompt.

Masks in the diagram drawn by hand, not by SAM.

# The 2026 toolbox

| You need ... | Reach for | You provide |
|---|---|---|
| boxes for *your* classes, fast, on a small device | Ultralytics YOLO (YOLO26, Jan 2026); RF-DETR (Roboflow, 2025) | labeled boxes - fine-tune ([51]) |
| boxes for anything you can name | open-vocabulary detectors: YOLO-World, Grounding DINO | class names as text |
| masks | SAM 2 (click or box, also video); SAM 3 (a typed concept) | a prompt |
| features for a new task | DINOv3, frozen ([51]) | a linear head |
| an answer in words - or boxes for a question | vision-language models (Gemini, Qwen-VL return boxes as text) | a question |

The pattern: big promptable models draft labels and build prototypes; a small fine-tuned model runs on the belt. Lists like this age fast - YOLO27 was announced in September 2026 and is not out yet. Check before you trust any "latest", including this one.

# The sorting line, completed

Cash in the running example - then zoom out on the whole chapter.

# One scenario, four output structures

The pomegranate line, end to end:

| **Question on the line** | **Task** | **Tool** |
|---|---|---|
| "what grade is this fruit?" | classification | [50]'s ResNet-18, fine-tuned in [51] |
| "is it grading fruit or belt?" | attribution | [51]'s Grad-CAM audit |
| "how many fruit, and where?" | detection | a fine-tuned YOLO |
| "where exactly is the bruise?" | segmentation | a small U-Net for the cutter |

Swap the nouns and it is the same story everywhere: roads on satellite tiles, tumors on MRI slices, cracks on factory parts.

One backbone vocabulary - conv, pool, skip, upsample - and the **output structure** turns it into whichever tool the problem needs.

# More output structures, same backbone

The task zoo had four outputs. Swap the head again and you get more:

- **Keypoints** - a fixed list of points per object. Google's MediaPipe hand model first *detects* the hand, then regresses **21 points** on it (knuckles, fingertips): $21 \times 3 = 63$ numbers, [01]'s regression. Gesture Snake's thumb direction could be a rule on two of those points.
- **Depth** - a *number* per pixel instead of a class: a U-Net-shaped network with a regression loss.
- **Text** - a caption, or an answer about the photo: a vision backbone feeding a language model (vision-language models, later in the course).

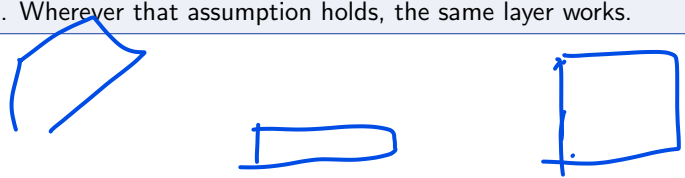
Detection = classification + regression in slots. Segmentation = classification per pixel. Keypoints = regression on a fixed list. Depth = regression per pixel. **Pick the output - the loss follows.**

# Same idea, other shapes: 1D and 3D

Nothing about convolution says "image". The window slides over whatever grid the data lives on:

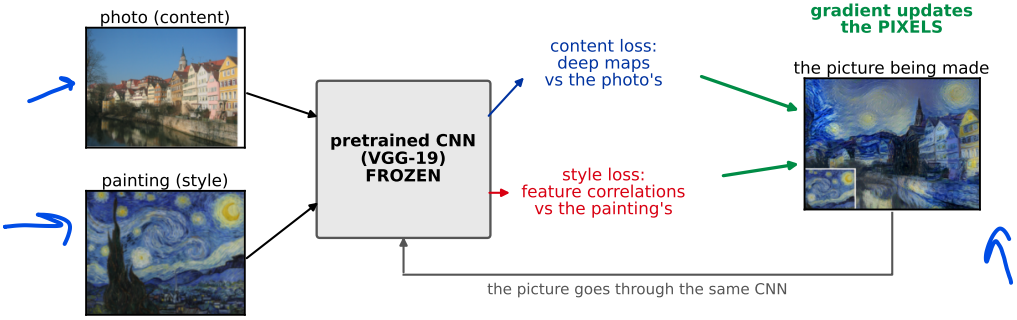
- **1D** - sequences: accelerometer traces for activity recognition, audio. [48] *opened* with a 1D kernel sliding over a sequence - that was the real thing, not a warm-up.
- **2D** - images: this whole chapter.
- **3D** - volumes: video (stacked frames), MRI and CT scans - the kernel slides in $x$, $y$ *and* $z$.

Convolution is a **structural prior** - "nearby means related, patterns repeat" - not an image trick. Wherever that assumption holds, the same layer works.



# CNNs can also paint: neural style transfer

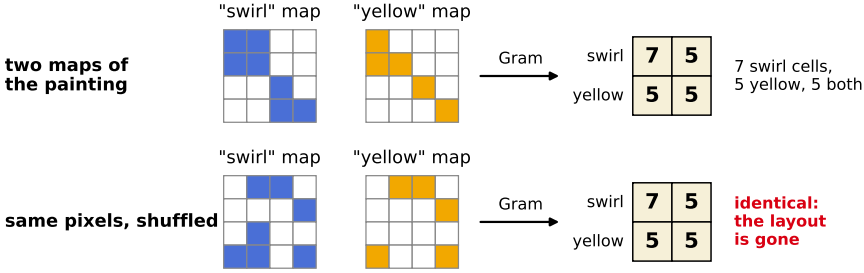
One more output structure: a **whole new picture**. Gatys et al. (2015) repainted a photo in Van Gogh's style with a network that was only ever trained to *classify*.




Nothing gets trained except the **picture**. Start from noise, run it through the frozen VGG-19 ([50]), measure two losses, and let gradient descent change its **pixels** - the same backprop as training, with the image as the parameter.

# What "style" means to a CNN

**Content:** the photo's deep-layer maps - they keep *what is where* (the layout), not exact pixels ([51]'s feature hierarchy). **Style:** for each layer, count which features fire *together* - the **Gram matrix** $G_{ij} = \sum_{\text{positions } p} F_i(p)\, F_j(p)$:




Summing over positions throws away *where* - the move [50]'s global average pooling made. What survives: which colours, textures and strokes go together. A fair definition of style.

Weight the two losses: more style, more painting. Since 2016 a network is *trained* to do it in one pass (Johnson et al.) - the real-time photo filters.

# CNNs can also invent: GANs



None of these people exist (StyleGAN, Karras et al. 2019).

**GANs**: two networks duel - a generator forges images, a discriminator calls fakes, both improve until the fakes pass (Goodfellow et al. 2014).

The generator upsamples noise into pixels with **transposed convolutions** - the stamps from the segmentation section.

Honest footnote: for image *generation*, **diffusion** has largely taken over ([50] epilogue) - GANs live on in upscaling and faces. Mechanics belong to the diffusion chapter (L27-L31).

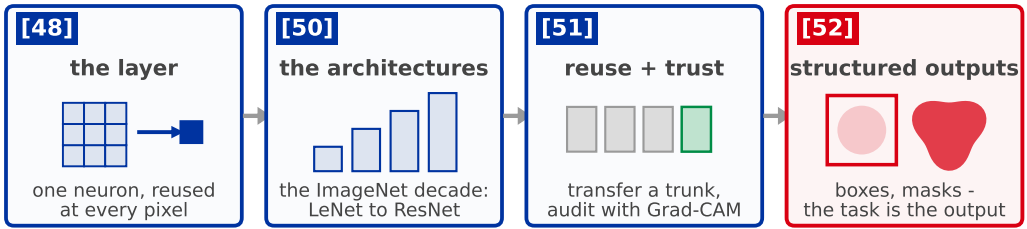
# The inductive-bias ledger

What this chapter really bought us - stated honestly:

- CNNs **assume** locality and translation equivariance ([48]'s two wishes: "nearby means related, a detector useful here is useful there").
- When the assumption is *true* - images, audio, volumes - it acts as **regularization by architecture**: win with *less data*.
- When it is *false*, it costs you: **tabular data** still belongs to trees and boosting (ch4, and [40]'s tabular caveat), and images where **global context** dominates strain the local prior.

There is no universally best architecture - only assumptions that **match or mismatch** the data. Choosing the bias *is* the modeling decision.

# The chapter in one strip




One layer $\rightarrow$ architectures $\rightarrow$ reuse and trust $\rightarrow$ structured outputs.
You can now read, adapt, audit and deploy the networks that do most of the world's looking.

# Hand-off: Vision Transformers

The [50] epilogue's loose thread, tied:

- **ViT** cuts the image into patches and feeds them to a transformer as a *sequence* - **no convolution at all** (Dosovitskiy et al., 2021).
- It **drops the convolutional prior** - and buys it back with **scale**: with enough pretraining data, attention *learns* locality instead of assuming it.
- Small data? The CNN prior still wins. Web-scale data? Attention takes the frontier. The ledger from two slides ago, live in 2026.

Attention, transformers and the GenAI stack get their own chapters - and the road there runs through RNNs, next.

## Recap

- The **output structure defines the task**: label → box → box per object → class per pixel.
- Detection = many **slots**, each a box + a **confidence** (predicted $c_o$): anchors at every grid cell, shifted and stretched by a head, in one pass. Then **IoU**, **NMS**, **mAP** (= [12] over boxes).
- **YOLO**: the centre's cell reports the object, computed from the whole photo; loss = box regression + confidence + class, added. **Label every object**.
- **Separable convs** make it fit on a belt camera ($\sim$23$\times$ cheaper).
- Segmentation: **U-Net** = down, up (**transposed convs**: stamps that add), skips; **dilation** for context. 2026: **SAM** masks on a click or a word.
- Convolution is a **prior with a domain** - and it can paint: style = the **Gram matrix**, which features fire together.

**Next:** the Gesture Snake practical - train a CNN on *your* webcam photos, audit it with Grad-CAM, and play with it. Then a new chapter: networks with **memory** - RNNs (L20).